

```
# — Install packages (Colab-safe)
import subprocess, sys

pkgs = ["pymc", "arviz", "scikit-learn"]
for pkg in pkgs:
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg], check=True)

print("\n✓ All packages installed")
```

✓ All packages installed

```
# — Clone GitHub repo and set paths
import os
from pathlib import Path

REPO_URL = "https://github.com/dakshini01/ProdFusion.git"
REPO_DIR = Path("/content/ProdFusion")

if not REPO_DIR.exists():
    os.system(f"git clone {REPO_URL}")
    print("✓ Repo cloned")
else:
    os.system("cd /content/ProdFusion && git pull")
    print("✓ Repo updated (git pull)")

MODEL_DIR = REPO_DIR / "models" # CSVs live here
OUTPUT_DIR = REPO_DIR / "output" # Figures saved here
BAYES_DIR = REPO_DIR / "bayesian" # Posteriors (.nc) saved here

OUTPUT_DIR.mkdir(exist_ok=True)
BAYES_DIR.mkdir(exist_ok=True)

print(f"  MODEL_DIR : {MODEL_DIR}")
print(f"  OUTPUT_DIR : {OUTPUT_DIR}")
print(f"  BAYES_DIR : {BAYES_DIR}")
print("\nCSV files found in models/:")
for f in sorted(MODEL_DIR.glob("*.csv")):
    print(f"  {f.name}")
```

✓ Repo cloned
 MODEL_DIR : /content/ProdFusion/models
 OUTPUT_DIR : /content/ProdFusion/output
 BAYES_DIR : /content/ProdFusion/bayesian

CSV files found in models/:

- bayesian_glm_ready.csv
- data_preprocessing_notes.csv
- d1m_ready.csv
- garment_bayesian_glm.csv
- garment_d1m.csv
- garment_full_cleaned.csv
- garment_test.csv
- garment_train.csv
- garment_val.csv
- scaler_params.csv
- test_ready.csv
- train_ready.csv
- val_ready.csv

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from pathlib import Path

import pymc as pm
import pytensor.tensor as pt
import arviz as az
from scipy.interpolate import BSpline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
```

```

mean_squared_error, mean_absolute_error, r2_score,
roc_auc_score, accuracy_score, precision_score, recall_score,
f1_score, balanced_accuracy_score, confusion_matrix
)

plt.rcParams["figure.dpi"] = 120
sns.set_theme(style="whitegrid")

print(f"PyMC version : {pm.__version__}")
print(f"ArviZ version : {az.__version__}")

```

```

PyMC version : 5.28.5
ArviZ version : 0.22.0

```

```

# — Load the modeling-ready CSV files produced by the cleaning notebook ———
REPO_DIR = Path("/content/ProdFusion")
MODEL_DIR = REPO_DIR / "models"

# Full dataset (already split – use split_code column: 0=train, 1=val, 2=test)
df_full = pd.read_csv(MODEL_DIR / "bayesian_glm_ready.csv")

print(f"Full dataset loaded: {df_full.shape}")
print(f"Columns ({len(df_full.columns)}): {df_full.columns.tolist()}")
print(f"\nSplit distribution (split_code):")
print(df_full["split_code"].value_counts().sort_index()
      .rename({0: "train", 1: "val", 2: "test"}).to_string())

```

```

Full dataset loaded: (1197, 44)
Columns (44): ['t', 'team', 'dept_bin', 'team_dept_idx', 'split_code', 'quarter', 'day_num', 'week_of_study', 'actual_productivi

Split distribution (split_code):
split_code
train      823
val        182
test       192

```

```

# — Load the modeling-ready CSV files produced by the cleaning notebook ———
REPO_DIR = Path("/content/ProdFusion")
MODEL_DIR = REPO_DIR / "models"

# Full dataset (already split – use split_code column: 0=train, 1=val, 2=test)
df_full = pd.read_csv(MODEL_DIR / "bayesian_glm_ready.csv")

print(f"Full dataset loaded: {df_full.shape}")
print(f"Columns ({len(df_full.columns)}): {df_full.columns.tolist()}")
print(f"\nSplit distribution (split_code):")
print(df_full["split_code"].value_counts().sort_index()
      .rename({0: "train", 1: "val", 2: "test"}).to_string())

```

```

Full dataset loaded: (1197, 44)
Columns (44): ['t', 'team', 'dept_bin', 'team_dept_idx', 'split_code', 'quarter', 'day_num', 'week_of_study', 'actual_productivi

Split distribution (split_code):
split_code
train      823
val        182
test       192

```

✓ 02 “ Bayesian Beta Regression | Garment Worker Productivity

This notebook builds a **hierarchical Beta regression** using **PyMC v5** to model garment worker productivity (a continuous outcome bounded in (0, 1)).

Key features:

- **Three nested models** (A → B → C) of increasing complexity, compared via PSIS-LOO
- **Monotone I-splines** to enforce a non-decreasing incentive effect
- **Interaction terms** (incentive → WIP, WIP → workers) in the full model
- **Partial pooling** of team-level intercepts via non-centered parameterization
- **Posterior predictive checks** and out-of-sample test evaluation

Prerequisite: `01_data_prep.ipynb` must have been run to produce `garments_clean.csv`, `data_splits.pkl`, and the `outputs/` directory.

Output directory: `E:\machine_learning\models\` will be created by this notebook to store NetCDF posterior files.

1. Imports

2. Load Processed Data

Please run the above two cells, wait for the runtime to restart, and then run the following cell to load the data.

```
# — Create train / validation / test sets using split_code —————

# Clean column names
df_full.columns = df_full.columns.str.strip()

print("Available columns:")
print(df_full.columns.tolist())

# Split using split_code
df_train = df_full[df_full["split_code"] == 0].copy()
df_val = df_full[df_full["split_code"] == 1].copy()
df_test = df_full[df_full["split_code"] == 2].copy()

# Combine train + validation for Bayesian model fitting
df_fit = pd.concat([df_train, df_val], ignore_index=True)

# Sort by time index column 't'
df_fit = df_fit.sort_values("t").reset_index(drop=True)
df_test = df_test.sort_values("t").reset_index(drop=True)

print(f"Train set: {len(df_train)} rows")
print(f"Validation set: {len(df_val)} rows")
print(f"Fit set: {len(df_fit)} rows | Test set: {len(df_test)} rows")

# Use transformed columns available in this CSV
summary_cols = [
    "actual_productivity",
    "log_incentive",
    "log_wip_imputed",
    "log_over_time",
    "log_workers",
    "log_smv",
    "targeted_productivity"
]

print(df_fit[summary_cols].describe())
```

```
Available columns:
['t', 'team', 'dept_bin', 'team_dept_idx', 'split_code', 'quarter', 'day_num', 'week_of_study', 'actual_productivity', 'producti
Train set: 823 rows
Validation set: 182 rows
Fit set: 1005 rows | Test set: 192 rows
```

	actual_productivity	log_incentive	log_wip_imputed	log_over_time \
count	1005.000000	1005.000000	1005.000000	1005.000000
mean	0.737175	1.922737	2.327467	8.005689
std	0.177906	1.984139	3.274419	1.392309
min	0.233705	0.000000	0.000000	0.000000
25%	0.650243	0.000000	0.000000	7.273093
50%	0.779792	0.000000	0.000000	8.314097
75%	0.850502	3.931826	6.703188	8.848078
max	1.120437	4.934474	10.048583	10.162809

	log_workers	log_smv	targeted_productivity
count	1005.000000	1005.000000	1005.000000
mean	3.247763	2.373257	0.731463
std	0.873655	0.904587	0.099323
min	0.693147	1.064711	0.070000
25%	2.197225	1.371181	0.700000
50%	3.526361	2.725235	0.750000

75%	4.043051	3.165053	0.800000
max	4.488636	3.999301	0.800000

3. Feature Preparation for PyMC

```
# Clip actual_productivity to strict (0,1) – Beta distribution requires open interval
EPS = 1e-4

y_fit = np.clip(df_fit["actual_productivity"].values, EPS, 1 - EPS)
y_test = np.clip(df_test["actual_productivity"].values, EPS, 1 - EPS)

# Use available transformed columns from bayesian_glm_ready.csv
# These replace the missing raw columns:
# incentive      -> log_incentive
# wip            -> log_wip_imputed
# over_time      -> log_over_time
# smv            -> log_smv
# no_of_workers  -> log_workers

feat_cols = [
    "log_incentive",
    "log_wip_imputed",
    "log_over_time",
    "log_smv",
    "log_workers",
    "no_of_style_change",
    "targeted_productivity"
]

scaler2 = StandardScaler()

X_fit_raw = df_fit[feat_cols].fillna(0).values
X_test_raw = df_test[feat_cols].fillna(0).values

X_fit = scaler2.fit_transform(X_fit_raw)
X_test = scaler2.transform(X_test_raw)

# Team indices
teams = sorted(df_fit["team"].unique())
team_map = {t: i for i, t in enumerate(teams)}

team_idx_fit = df_fit["team"].map(team_map).values.astype(int)
team_idx_test = df_test["team"].map(team_map).fillna(0).values.astype(int)

n_teams = len(teams)
team_labels = np.array(teams)

# Department variable
# Your CSV has dept_bin, not department
dept_fit = df_fit["dept_bin"].values.astype(float)
dept_test = df_test["dept_bin"].values.astype(float)

# WIP missing indicator
# Your CSV has wip_missing_flag, not wip_missing
wip_missing_fit = df_fit["wip_missing_flag"].values.astype(float)
wip_missing_test = df_test["wip_missing_flag"].values.astype(float)

# Column indices
COL = {name: i for i, name in enumerate(feat_cols)}

# Old-name aliases so later model code still works
COL["incentive"] = COL["log_incentive"]
COL["wip"] = COL["log_wip_imputed"]
COL["over_time"] = COL["log_over_time"]
COL["smv"] = COL["log_smv"]
COL["no_of_workers"] = COL["log_workers"]

inc_idx = COL["incentive"]
wip_idx = COL["wip"]
ot_idx = COL["over_time"]
smv_idx = COL["smv"]
nw_idx = COL["no_of_workers"]
tp_idx = COL["targeted_productivity"]
style_idx = COL["no_of_style_change"]
```

```

n_fit = len(df_fit)
n_test = len(df_test)

print("Feature columns used:", feat_cols)
print(f"Teams: {n_teams}, Team index range: {team_idx_fit.min()}-{team_idx_fit.max()}")
print(f"X_fit shape: {X_fit.shape} | X_test shape: {X_test.shape}")

```

```

Feature columns used: ['log_incentive', 'log_wip_imputed', 'log_over_time', 'log_smv', 'log_workers', 'no_of_style_change', 'tar
Teams: 12, Team index range: 0-11
X_fit shape: (1005, 7) | X_test shape: (192, 7)

```

4. I-Spline Basis for Monotone Incentive Effect

I-splines (integrated B-splines) guarantee a **monotone non-decreasing** effect when all weights are non-negative. This encodes the prior belief that higher incentive pay cannot harm productivity.

```

def b_spline_basis(x, knots, degree=3):
    """B-spline basis matrix via scipy."""
    t = np.concatenate([
        np.repeat(knots[0], degree),
        knots,
        np.repeat(knots[-1], degree),
    ])
    n_basis = len(t) - degree - 1
    B = np.zeros((len(x), n_basis))
    for i in range(n_basis):
        c = np.zeros(n_basis); c[i] = 1.0
        spl = BSpline(t, c, degree)
        B[:, i] = spl(x)
    return B

def i_spline_basis(x, knots, degree=3):
    """Integrated B-splines -> monotone increasing with non-negative weights."""
    B = b_spline_basis(x, knots, degree)
    I = np.cumsum(B[:, :-1], axis=1)[:, :-1]
    return I[:, :-1] # drop last (identically 1 everywhere)

# Build on standardized incentive values
incentive_std_fit = X_fit[:, COL['incentive']]
incentive_std_test = X_test[:, COL['incentive']]

knots_inc = np.quantile(incentive_std_fit, np.linspace(0, 1, 8))
I_fit = i_spline_basis(incentive_std_fit, knots_inc).astype(np.float32)
I_test = i_spline_basis(incentive_std_test, knots_inc).astype(np.float32)
n_spline = I_fit.shape[1]

print(f"I-spline basis shape: {I_fit.shape} (n_obs x n_spline)")

I-spline basis shape: (1005, 9) (n_obs x n_spline)

```

5. Model A – Baseline Beta Regression

A simple global Beta regression with linear effects only – no team structure, no splines. This serves as a sanity check and comparison baseline.

Sampling: 500 draws, 500 tune steps, 2 chains.

```

coords_A = {"obs": np.arange(len(y_fit))}

with pm.Model(coords=coords_A) as model_A:
    # Data containers
    incentive_data = pm.Data("incentive", X_fit[:, COL['incentive']], dims="obs")
    wip_data = pm.Data("wip", X_fit[:, COL['wip']], dims="obs")
    ot_data = pm.Data("over_time", X_fit[:, COL['over_time']], dims="obs")
    smv_data = pm.Data("smv", X_fit[:, COL['smv']], dims="obs")
    nw_data = pm.Data("nw", X_fit[:, COL['no_of_workers']], dims="obs")
    dept_data = pm.Data("dept", dept_fit, dims="obs")
    wip_miss_data = pm.Data("wip_miss", wip_missing_fit, dims="obs")

    # Priors
    intercept = pm.Normal("intercept", mu=0, sigma=2)

```

```

b_incentive = pm.Normal("b_incentive", mu=0, sigma=1)
b_wip       = pm.Normal("b_wip",      mu=0, sigma=1)
b_ot        = pm.Normal("b_ot",       mu=0, sigma=1)
b_smv       = pm.Normal("b_smv",      mu=0, sigma=1)
b_nw        = pm.Normal("b_nw",      mu=0, sigma=1)
b_dept      = pm.Normal("b_dept",     mu=0, sigma=1)
b_wip_miss  = pm.Normal("b_wip_miss", mu=0, sigma=1)
nu          = pm.Exponential("nu",    lam=0.1)

eta = (intercept
      + b_incentive * incentive_data
      + b_wip       * wip_data
      + b_ot        * ot_data
      + b_smv       * smv_data
      + b_nw        * nw_data
      + b_dept      * dept_data
      + b_wip_miss  * wip_miss_data)

mu_A = pm.Deterministic("mu_A", pm.math.invlogit(eta), dims="obs")
y_obs = pm.Beta("y_obs", mu=mu_A, nu=nu, observed=y_fit, dims="obs")

idata_A = pm.sample(500, tune=500, chains=2, target_accept=0.9,
                    idata_kwargs={"log_likelihood": True},
                    random_seed=42, progressbar=True)

print("\nModel A \u2014 Baseline Beta Regression")
print(az.summary(idata_A, var_names=["intercept", "b_incentive", "b_wip", "b_ot", "b_smv", "b_nw", "b_dept", "b_wip_miss", "nu"]))

```

Progress	Draw	Divergences	Step size	Grad evals	Speed	Elapsed	Remaining
	1000	0	0.043	79	15.43 draws/s	0:01:04	0:00:00
	1000	0	0.039	103	9.80 draws/s	0:01:42	0:00:00

Model A — Baseline Beta Regression

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
intercept	1.178	0.623	0.035	2.337	0.031	0.022	421.0	
b_incentive	0.731	0.042	0.658	0.813	0.001	0.001	922.0	
b_wip	0.051	0.033	-0.010	0.117	0.001	0.001	912.0	
b_ot	-0.138	0.033	-0.204	-0.080	0.001	0.001	946.0	
b_smv	-0.073	0.109	-0.262	0.150	0.003	0.003	1035.0	
b_nw	0.633	0.089	0.462	0.795	0.003	0.003	921.0	
b_dept	-1.099	0.631	-2.223	0.088	0.030	0.021	440.0	
b_wip_miss	1.400	0.633	0.197	2.519	0.031	0.023	407.0	
nu	5.019	0.207	4.627	5.384	0.008	0.006	784.0	

	ess_tail	r_hat
intercept	489.0	1.00
b_incentive	677.0	1.00
b_wip	810.0	1.00
b_ot	767.0	1.00
b_smv	790.0	1.01
b_nw	752.0	1.00
b_dept	503.0	1.00
b_wip_miss	539.0	1.00
nu	773.0	1.00

6. Model B — Hierarchical Beta Regression with Team Random Effects

Extends Model A by adding **partial pooling** of team intercepts via a **non-centered parameterization** (avoids funnel geometry in HMC).

Sampling: 1000 draws, 1000 tune steps, 4 chains.

```

team_coords = [f"team_{t}" for t in teams]
coords_B = {"obs": np.arange(len(y_fit)), "team": team_coords}

with pm.Model(coords=coords_B) as model_B:
    # Data
    incentive_data = pm.Data("incentive", X_fit[:, COL['incentive']], dims="obs")
    wip_data       = pm.Data("wip",      X_fit[:, COL['wip']],      dims="obs")
    ot_data        = pm.Data("over_time", X_fit[:, COL['over_time']], dims="obs")
    smv_data       = pm.Data("smv",      X_fit[:, COL['smv']],      dims="obs")
    nw_data        = pm.Data("nw",      X_fit[:, COL['no_of_workers']], dims="obs")
    dept_data      = pm.Data("dept",    dept_fit,                  dims="obs")
    wip_miss_data  = pm.Data("wip_miss", wip_missing_fit,          dims="obs")
    team_data      = pm.Data("team_idx", team_idx_fit,             dims="obs")

```

```

# Fixed effects
b_incentive = pm.Normal("b_incentive", mu=0, sigma=1)
b_wip       = pm.Normal("b_wip",      mu=0, sigma=1)
b_ot        = pm.Normal("b_ot",       mu=0, sigma=1)
b_smv       = pm.Normal("b_smv",      mu=0, sigma=1)
b_nw        = pm.Normal("b_nw",       mu=0, sigma=1)
b_dept      = pm.Normal("b_dept",     mu=0, sigma=1)
b_wip_miss  = pm.Normal("b_wip_miss", mu=0, sigma=1)

# Hierarchical team intercepts (non-centered)
mu_team     = pm.Normal("mu_team",    mu=0, sigma=2)
sigma_team  = pm.Exponential("sigma_team", lam=1)
z_team      = pm.Normal("z_team",     mu=0, sigma=1, dims="team")
alpha_team  = pm.Deterministic("alpha_team",
                               mu_team + z_team * sigma_team, dims="team")

# Precision
nu = pm.Exponential("nu", lam=0.1)

eta = (alpha_team[team_data]
      + b_incentive * incentive_data
      + b_wip       * wip_data
      + b_ot        * ot_data
      + b_smv       * smv_data
      + b_nw        * nw_data
      + b_dept      * dept_data
      + b_wip_miss  * wip_miss_data)

mu_B = pm.Deterministic("mu_B", pm.math.invlogit(eta), dims="obs")
pm.Beta("y_obs", mu=mu_B, nu=nu, observed=y_fit, dims="obs")

idata_B = pm.sample(1000, tune=1000, chains=4, target_accept=0.9,
                    idata_kwargs={"log_likelihood": True},
                    random_seed=42, progressbar=True)

print("\nModel B \u2014 Hierarchical Team Effects")
print(az.summary(idata_B, var_names=["mu_team", "sigma_team", "b_incentive", "b_wip", "b_ot", "nu"]))

```

Progress	Draw	Divergences	Step size	Grad evals	Speed	Elapsed	Remaining
	2000	0	0.039	63	15.64 draws/s	0:02:07	0:00:00
	2000	0	0.046	127	7.95 draws/s	0:04:11	0:00:00
	2000	0	0.041	127	5.53 draws/s	0:06:01	0:00:00
	2000	0	0.037	191	4.11 draws/s	0:08:06	0:00:00

Model B – Hierarchical Team Effects

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk \
mu_team	1.122	0.695	-0.182	2.402	0.016	0.011	1819.0
sigma_team	0.339	0.092	0.202	0.509	0.003	0.003	1051.0
b_incentive	0.654	0.045	0.573	0.739	0.001	0.001	4742.0
b_wip	0.046	0.034	-0.014	0.113	0.001	0.001	4351.0
b_ot	-0.115	0.032	-0.175	-0.056	0.001	0.001	4108.0
nu	5.559	0.247	5.105	6.029	0.004	0.004	4295.0

	ess_tail	r_hat
mu_team	2153.0	1.00
sigma_team	1573.0	1.01
b_incentive	2819.0	1.00
b_wip	2803.0	1.00
b_ot	2773.0	1.00
nu	2870.0	1.00

7. Model C – Hierarchical + Monotone Spline on Incentive + Interactions

The full model replaces the linear incentive term with the **I-spline basis** (monotone constraint enforced by `HalfNormal` weights) and adds two **interaction terms**:

- `incentive_std ~ wip_std` – does incentive effectiveness depend on work-in-progress load?
- `wip_std ~ no_of_workers_std` – do more workers help more when WIP is high?

Sampling: 1000 draws, 1000 tune, 4 chains, `target_accept=0.92`.

```

# Pre-compute centered interaction terms
inc_x_wip = (X_fit[:, COL['incentive']] * X_fit[:, COL['wip']])
wip_x_nw  = (X_fit[:, COL['wip']]      * X_fit[:, COL['no_of_workers']])

```

```

coords_C = {"obs": np.arange(len(y_fit)), "team": team_coords, "spline": np.arange(n_spline)}

with pm.Model(coords=coords_C) as model_C:
    # Data
    I_data = pm.Data("I_spline", I_fit, dims=("obs", "spline"))
    wip_data = pm.Data("wip", X_fit[:, COL['wip']], dims="obs")
    ot_data = pm.Data("over_time", X_fit[:, COL['over_time']], dims="obs")
    smv_data = pm.Data("smv", X_fit[:, COL['smv']], dims="obs")
    nw_data = pm.Data("nw", X_fit[:, COL['no_of_workers']], dims="obs")
    dept_data = pm.Data("dept", dept_fit, dims="obs")
    wip_miss_d = pm.Data("wip_miss", wip_missing_fit, dims="obs")
    team_data = pm.Data("team_idx", team_idx_fit, dims="obs")
    ix_inc_wip = pm.Data("ix_inc_wip", inc_x_wip, dims="obs")
    ix_wip_nw = pm.Data("ix_wip_nw", wip_x_nw, dims="obs")

    # Monotone spline weights (non-negative -> monotone increasing incentive effect)
    w_spline = pm.HalfNormal("w_spline", sigma=1, dims="spline")
    spline_effect = pm.Deterministic("spline_effect",
                                     pt.dot(I_data, w_spline), dims="obs")

    # Fixed effects
    b_wip = pm.Normal("b_wip", mu=0, sigma=1)
    b_ot = pm.Normal("b_ot", mu=0, sigma=1)
    b_smv = pm.Normal("b_smv", mu=0, sigma=1)
    b_nw = pm.Normal("b_nw", mu=0, sigma=1)
    b_dept = pm.Normal("b_dept", mu=0, sigma=1)
    b_wip_miss = pm.Normal("b_wip_miss", mu=0, sigma=1)

    # Interaction priors (tighter)
    b_inc_wip = pm.Normal("b_inc_wip", mu=0, sigma=0.5)
    b_wip_nw = pm.Normal("b_wip_nw", mu=0, sigma=0.5)

    # Hierarchical team intercepts (non-centered)
    mu_team = pm.Normal("mu_team", mu=0, sigma=2)
    sigma_team = pm.Exponential("sigma_team", lam=1)
    z_team = pm.Normal("z_team", mu=0, sigma=1, dims="team")
    alpha_team = pm.Deterministic("alpha_team",
                                   mu_team + z_team * sigma_team, dims="team")

    nu = pm.Exponential("nu", lam=0.1)

    eta = (alpha_team[team_data]
           + spline_effect
           + b_wip * wip_data
           + b_ot * ot_data
           + b_smv * smv_data
           + b_nw * nw_data
           + b_dept * dept_data
           + b_wip_miss * wip_miss_d
           + b_inc_wip * ix_inc_wip
           + b_wip_nw * ix_wip_nw)

    mu_C = pm.Deterministic("mu_C", pm.math.clip(pm.math.invlogit(eta), EPS, 1 - EPS), dims="obs")
    pm.Beta("y_obs", mu=mu_C, nu=nu, observed=y_fit, dims="obs")

    idata_C = pm.sample(1000, tune=1500, chains=4, target_accept=0.95,
                        idata_kwargs={"log_likelihood": True},
                        random_seed=42, progressbar=True)

print("\nModel C \u2014 Hierarchical + I-Spline + Interactions")
print(az.summary(idata_C, var_names=["mu_team", "sigma_team", "w_spline", "b_wip", "b_ot", "b_inc_wip", "b_wip_nw", "nu"]))

```


Progress	Draw	Divergences	Step size	Grad evals	Speed	Elapsed	Remaining
	2500	0	0.009	255	3.34 draws/s	0:12:28	0:00:00
	2500	0	0.010	511	1.65 draws/s	0:25:11	0:00:00
	2500	0	0.009	511	1.11 draws/s	0:37:26	0:00:00
	2500	0	0.012	511	1.22 s/draw	0:50:38	0:00:00

Model C – Hierarchical + I-Spline + Interactions

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	\
mu_team	-1.550	1.056	-3.526	0.374	0.019	0.016	3252.0	
sigma_team	0.174	0.055	0.083	0.279	0.001	0.001	1553.0	
w_spline[0]	0.646	0.513	0.000	1.587	0.008	0.008	2921.0	
w_spline[1]	0.649	0.523	0.000	1.611	0.008	0.010	2825.0	
w_spline[2]	0.664	0.531	0.000	1.633	0.007	0.009	3166.0	
w_spline[3]	0.675	0.516	0.001	1.599	0.007	0.009	4132.0	
w_spline[4]	0.058	0.055	0.000	0.158	0.001	0.001	3288.0	
w_spline[5]	0.064	0.060	0.000	0.175	0.001	0.001	3782.0	
w_spline[6]	0.772	0.146	0.511	1.063	0.002	0.002	3553.0	
w_spline[7]	0.358	0.182	0.006	0.653	0.003	0.003	2596.0	
w_spline[8]	2.271	0.253	1.792	2.754	0.004	0.004	3688.0	
b_wip	0.223	0.091	0.056	0.402	0.002	0.001	2913.0	
b_ot	-0.062	0.031	-0.119	-0.003	0.000	0.001	5466.0	
b_inc_wip	-0.058	0.042	-0.137	0.023	0.001	0.001	5075.0	
b_wip_nw	-0.235	0.100	-0.427	-0.048	0.002	0.002	2878.0	
nu	6.827	0.298	6.292	7.398	0.004	0.005	5943.0	

	ess_tail	r_hat
mu_team	2934.0	1.0
sigma_team	1901.0	1.0
w_spline[0]	1833.0	1.0
w_spline[1]	1493.0	1.0
w_spline[2]	1232.0	1.0
w_spline[3]	1972.0	1.0
w_spline[4]	2204.0	1.0
w_spline[5]	2752.0	1.0
w_spline[6]	2581.0	1.0
w_spline[7]	1405.0	1.0
w_spline[8]	2729.0	1.0
b_wip	2658.0	1.0
b_ot	2840.0	1.0
b_inc_wip	2937.0	1.0
b_wip_nw	2729.0	1.0
nu	2755.0	1.0

8. PSIS-LOO Model Comparison

Pareto-Smoothed Importance Sampling Leave-One-Out cross-validation (PSIS-LOO) is a fast approximation to exact LOO-CV. Models are stacked (rather than pseudo-BMA weighted) for robustness.

```
OUTPUT_DIR.mkdir(exist_ok=True)

comparison = az.compare(
    {
        "Model_A_baseline": idata_A,
        "Model_B_hierarchical": idata_B,
        "Model_C_std_interaction": idata_C,
    },
    ic="loo",
    method="stacking",
    scale="log"
)

print("\nPSIS-LOO Model Comparison:")
print(comparison)

fig, ax = plt.subplots(figsize=(8, 4))
az.plot_compare(comparison, insample_dev=False, ax=ax)

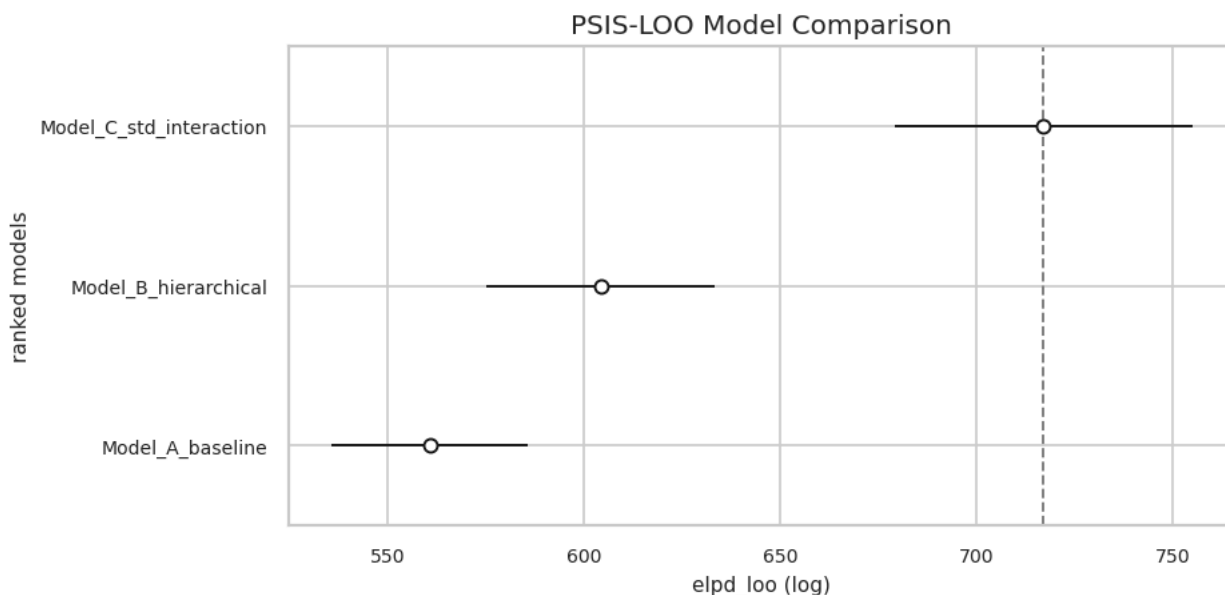
ax.set_title("PSIS-LOO Model Comparison")
plt.tight_layout()

plt.savefig(OUTPUT_DIR / "02_model_comparison_loo.png", dpi=120)
plt.show()
```

PSIS-LOO Model Comparison:

	rank	elpd_loo	p_loo	elpd_diff	\
Model_C_std_interaction	0	717.233056	21.984683	0.000000	
Model_B_hierarchical	1	604.385890	23.941670	112.847166	
Model_A_baseline	2	560.858593	10.957067	156.374463	

	weight	se	dse	warning	scale
Model_C_std_interaction	9.632007e-01	37.783908	0.000000	False	log
Model_B_hierarchical	3.679934e-02	28.963490	17.351289	False	log
Model_A_baseline	5.218048e-15	24.974534	20.232775	False	log



9. Posterior Coefficient Plot for Best Model (Model C)

Forest plots show the **94% HDI** (higher-density interval) for all fixed effects and team-level intercepts.

```
# Check available variables in Model C
print("Available posterior variables in idata_C:")
print(list(idata_C.posterior.data_vars))
```

```
# Forest plot of fixed effects
possible_effects = [
    # New corrected Model C names
    "b_inc",
    "b_wip",
    "b_ot",
    "b_smv",
    "b_nw",
    "b_tp",
    "b_dept",
    "b_wm",
    "b_ix_inc_wip",
    "b_ix_wip_nw",

    # Old Model C names
    "b_wip_miss",
    "b_inc_wip",
    "b_wip_nw"
]
```

```
var_names_available = [
    v for v in possible_effects
    if v in idata_C.posterior.data_vars
]
```

```
print("\nVariables used in forest plot:")
print(var_names_available)
```

```
az.plot_forest(
    idata_C,
```

```
        var_names=var_names_available,
        combined=True,
        hdi_prob=0.94,
        figsize=(8, 5)
    )

    plt.title("Posterior Effect Sizes – Model C (94% HDI)")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "02_posterior_effects.png", dpi=120)
    plt.show()

# Team-level intercepts
if "alpha_team" in idata_C.posterior.data_vars:
    az.plot_forest(
        idata_C,
        var_names=["alpha_team"],
        combined=True,
        hdi_prob=0.94,
        figsize=(8, 6)
    )

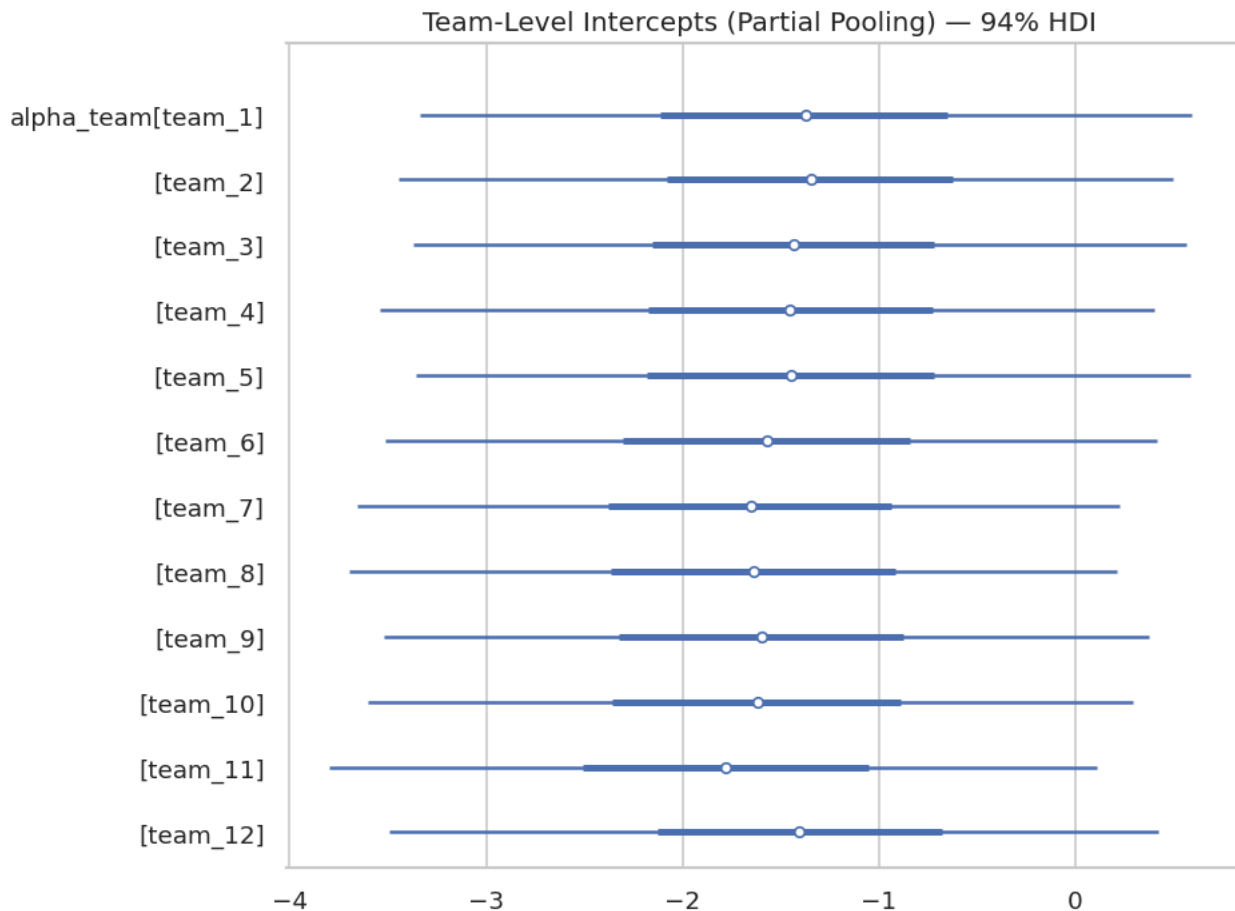
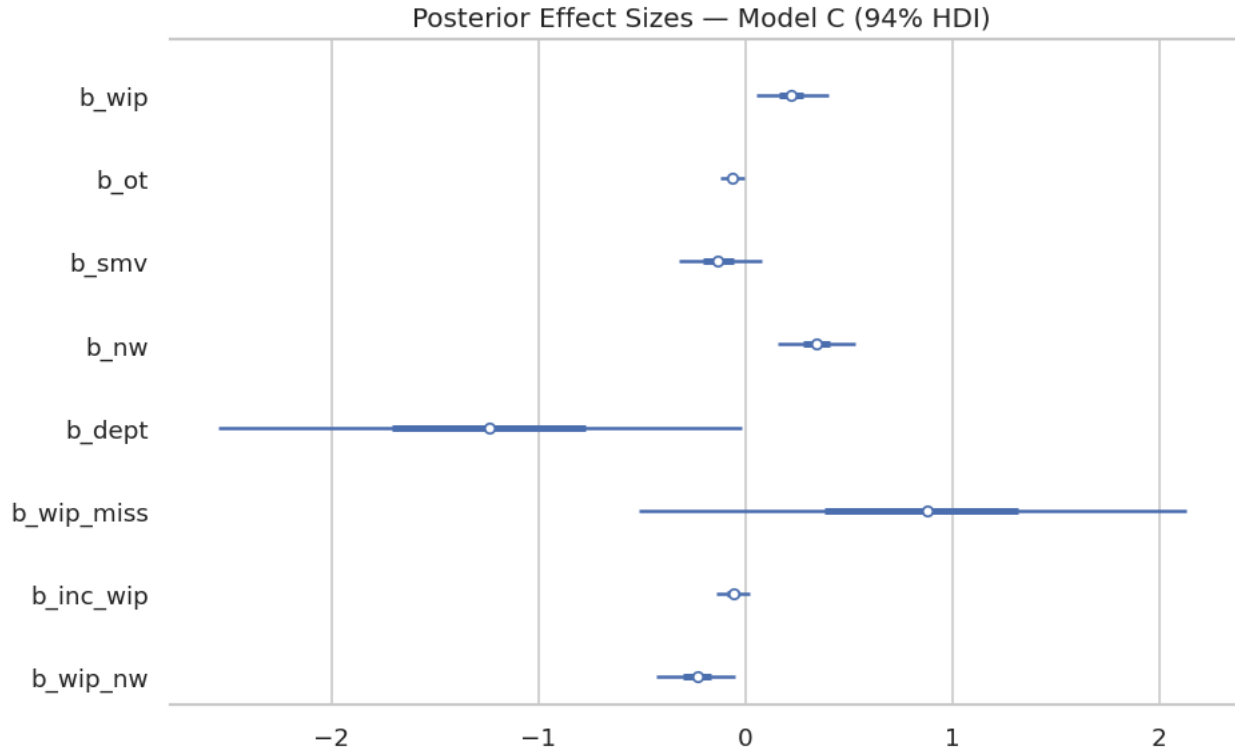
    plt.title("Team-Level Intercepts (Partial Pooling) – 94% HDI")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "02_team_intercepts.png", dpi=120)
    plt.show()
else:
    print("alpha_team not found in idata_C.")
```

Available posterior variables in idata_C:

['alpha_team', 'b_dept', 'b_inc_wip', 'b_nw', 'b_ot', 'b_smv', 'b_wip', 'b_wip_miss', 'b_wip_nw', 'mu_C', 'mu_team', 'nu', 'sigm

Variables used in forest plot:

['b_wip', 'b_ot', 'b_smv', 'b_nw', 'b_dept', 'b_wip_miss', 'b_inc_wip', 'b_wip_nw']



10. Monotone Spline Visualization

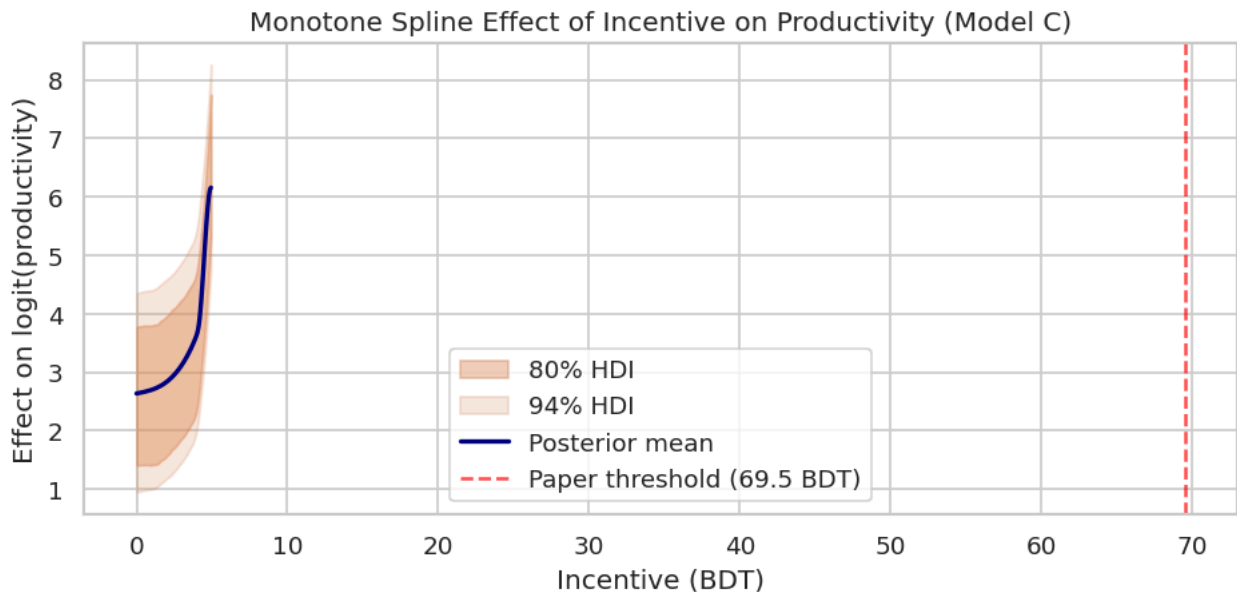
The estimated incentive effect is evaluated over a dense grid of incentive values, projecting I-spline weights onto the basis functions. The monotone constraint guarantees the curve is non-decreasing.

```
# Reconstruct spline effect over a fine grid
inc_grid_std = np.linspace(incentive_std_fit.min(), incentive_std_fit.max(), 100)
I_grid = i_spline_basis(inc_grid_std, knots_inc).astype(np.float32)

# Extract posterior w_spline samples
w_samples = idata_C.posterior["w_spline"].values.reshape(-1, n_spline) # (S, n_spline)
spline_grid_samples = w_samples @ I_grid.T # (S, 100)

# Convert to original incentive scale for x-axis
inc_grid_orig = inc_grid_std * scaler2.scale_[COL['incentive']] + scaler2.mean_[COL['incentive']]

fig, ax = plt.subplots(figsize=(8, 4))
az.plot_hdi(inc_grid_orig, spline_grid_samples, hdi_prob=0.80,
            fill_kwargs={"alpha": 0.4, "label": "80% HDI"}, ax=ax)
az.plot_hdi(inc_grid_orig, spline_grid_samples, hdi_prob=0.94,
            fill_kwargs={"alpha": 0.2, "label": "94% HDI"}, ax=ax)
ax.plot(inc_grid_orig, spline_grid_samples.mean(axis=0), color="navy", lw=2, label="Posterior mean")
ax.axvline(69.5, color="red", ls="--", alpha=0.7, label="Paper threshold (69.5 BDT)")
ax.set_xlabel("Incentive (BDT)")
ax.set_ylabel("Effect on logit(productivity)")
ax.set_title("Monotone Spline Effect of Incentive on Productivity (Model C)")
ax.legend()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "02_incentive_spline.png", dpi=120)
plt.show()
```



11. Posterior Predictive Check

Two complementary diagnostics:

1. **PPC density overlay** â€” simulated replications vs. observed distribution
2. **LOO-PIT** â€” probability integral transform using LOO predictives; uniform = well-calibrated

```
with model_C:
    ppc_C_samples = pm.sample_posterior_predictive(idata_C, random_seed=42)

# Manually add the posterior_predictive group to idata_C
# ArviZ InferenceData objects have an add_groups method for this purpose
if not hasattr(idata_C, 'posterior_predictive'):
    idata_C.add_groups(posterior_predictive=ppc_C_samples.posterior_predictive)
```

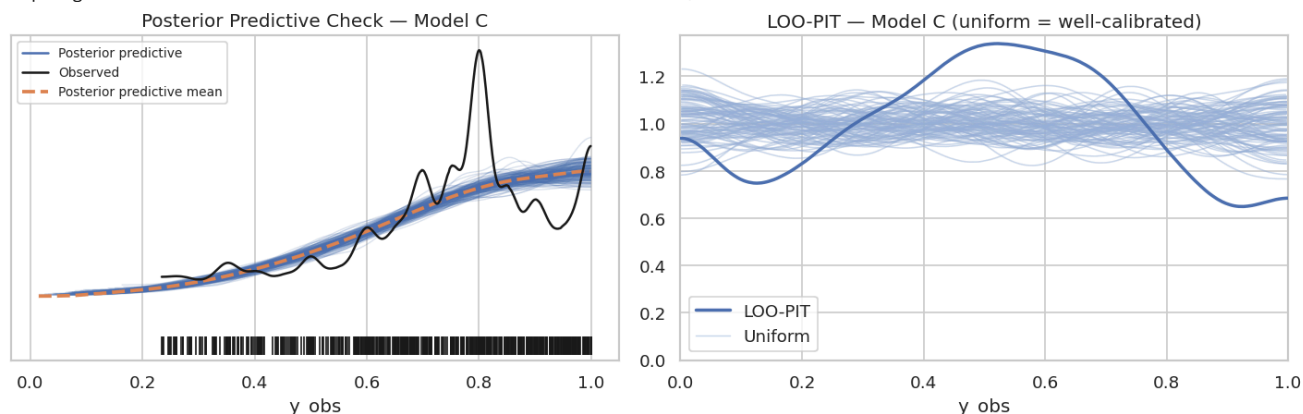
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# PPC density overlay
az.plot_ppc(idata_C, observed_rug=True, ax=axes[0], num_pp_samples=200)
axes[0].set_title("Posterior Predictive Check — Model C")

# LOO-PIT
az.plot_loo_pit(idata=idata_C, y="y_obs", ax=axes[1])
axes[1].set_title("LOO-PIT — Model C (uniform = well-calibrated)")

plt.tight_layout()
plt.savefig(OUTPUT_DIR / "02_ppc_and_loopit.png", dpi=120)
plt.show()
```

Sampling ... 100% 0:00:00 / 0:00:01



12. Test-Set Predictions

Out-of-sample evaluation on the held-out test set using `pm.set_data()` to swap in test features, then `sample_posterior_predictive` to draw from the posterior predictive distribution.

```
# Out-of-sample prediction for Model C
# Recompute test features
inc_x_wip_test = X_test[:, COL['incentive']] * X_test[:, COL['wip']]
wip_x_nw_test = X_test[:, COL['wip']] * X_test[:, COL['no_of_workers']]

# Define new coordinates for the test set 'obs' dimension
coords_test = {"obs": np.arange(len(y_test))}

with model_C:
    pm.set_data({
        "I_spline": I_test,
        "wip": X_test[:, COL['wip']],
        "over_time": X_test[:, COL['over_time']],
        "smv": X_test[:, COL['smv']],
        "nw": X_test[:, COL['no_of_workers']],
        "dept": dept_test,
        "wip_miss": wip_missing_test,
        "team_idx": team_idx_test,
        "ix_inc_wip": inc_x_wip_test,
        "ix_wip_nw": wip_x_nw_test,
    }, coords=coords_test)
    ppc_test = pm.sample_posterior_predictive(idata_C, random_seed=42,
                                             var_names=["mu_C", "y_obs"], return_inferencedata=False)

mu_test_samples = ppc_test["mu_C"].reshape(-1, len(y_test))
mu_test_mean = mu_test_samples.mean(axis=0)

# MAE and RMSE
mae = np.mean(np.abs(mu_test_mean - y_test))
rmse = np.sqrt(np.mean((mu_test_mean - y_test)**2))
```

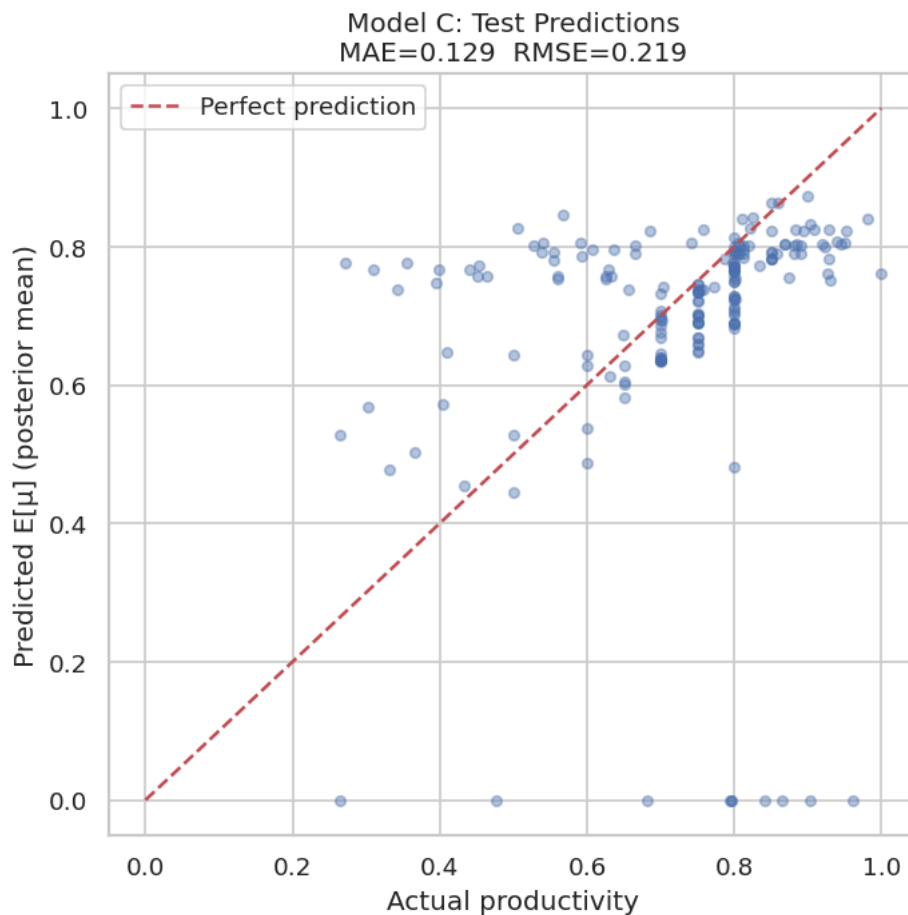
```

print(f"Test MAE: {mae:.4f}")
print(f"Test RMSE: {rmse:.4f}")

# Scatter: predicted vs actual
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(y_test, mu_test_mean, alpha=0.4, s=20)
ax.plot([0,1],[0,1], 'r--', lw=1.5, label="Perfect prediction")
ax.set_xlabel("Actual productivity")
ax.set_ylabel("Predicted E[\u03bc] (posterior mean)")
ax.set_title(f"Model C: Test Predictions\nMAE={mae:.3f} RMSE={rmse:.3f}")
ax.legend()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "02_test_predictions.png", dpi=120)
plt.show()

```

Sampling ... 100% 0:00:00 / 0:00:00
 Test MAE: 0.1292
 Test RMSE: 0.2193



13. Save Posteriors

```

idata_A.to_netcdf(str(MODEL_DIR / "model_A_baseline.nc"))
idata_B.to_netcdf(str(MODEL_DIR / "model_B_hierarchical.nc"))
idata_C.to_netcdf(str(MODEL_DIR / "model_C_spline_interaction.nc"))

# Also save the model comparison table
comparison.to_csv(MODEL_DIR / "02_loo_comparison.csv")

# Download the comparison table
from google.colab import files
files.download(MODEL_DIR / "02_loo_comparison.csv")

print("Posteriors saved to models/")
print("Figures saved to outputs/")
print("\nKey findings:")
print(f"  Best model (LOO): {comparison.index[0]}")
print(f"  Test MAE: {mae:.4f}")

```

```
print(f" Test RMSE: {rmse: .4f}")
```

Posteriors saved to models/
Figures saved to outputs/

Key findings:

Best model (LOO): Model_C_std_interaction
Test MAE: 0.1292
Test RMSE: 0.2193
→ Ready for 03_kalman_dlm.ipynb and 04_validation.ipynb

```
# — Git push back to GitHub —————
from getpass import getpass
from urllib.parse import quote
import subprocess

%cd /content/ProdFusion

!git config --global user.name "dakshini01"
!git config --global user.email "dakshini01@gmail.com"

# FIXED: Changed file name to match the '02_' prefix, and added tracking for the .nc files
!git add bayesian/ output/ models/02_loo_comparison.csv models/*.nc

!git commit -m "Add Bayesian model posteriors, 02 comparison CSV, and output figures" || echo "Nothing to commit"

token = getpass("Paste your GitHub token: ")
safe_tok = quote(token, safe="")
repo_url = f"https://dakshini01:{safe_tok}@github.com/dakshini01/ProdFusion.git"

pull = subprocess.run(
    ["git", "pull", "--rebase", repo_url, "main"],
    text=True, capture_output=True
)
print("PULL:", pull.returncode)
print(pull.stdout)

if pull.returncode == 0:
    push = subprocess.run(
        ["git", "push", repo_url, "main"],
        text=True, capture_output=True
    )
    print("PUSH:", push.returncode)
    if push.returncode == 0:
        print("✓ Results pushed to GitHub successfully")
    else:
        print("X Push failed:", push.stderr.replace(token, "****"))
else:
    print("X Pull failed")
```

```
/content/ProdFusion
[main 3093a6e] Add Bayesian model posteriors, 02 comparison CSV, and output figures
4 files changed, 4 insertions(+)
create mode 100644 models/02_loo_comparison.csv
create mode 100644 models/model_A_baseline.nc
create mode 100644 models/model_B_hierarchical.nc
create mode 100644 models/model_C_spline_interaction.nc
Paste your GitHub token: .....
PULL: 0
Current branch main is up to date.

PUSH: 0
✓ Results pushed to GitHub successfully
```